

Backstage | CON
NORTH AMERICA

Treating Backstage Like a Product

Observability with OpenTelemetry

Ekansh Gupta

Software Engineer, Bindplane (Dynatrace)

Everything in this talk runs on this laptop.

Before any slides, three numbers from a running portal.

100%

of runs of one template fail.
Every single one.

104

lookups in one hour of catalog entities
that do not exist.

307

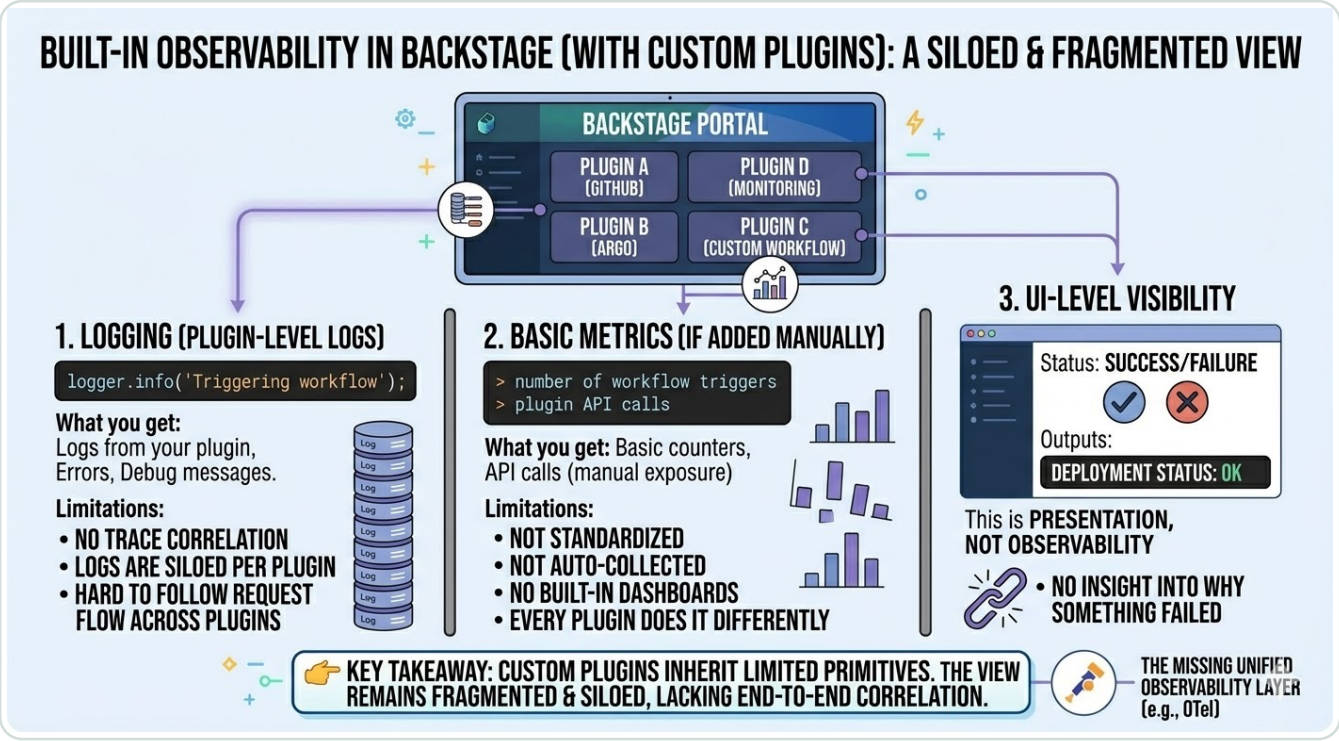
TechDocs pages requested in one hour
that were never built.

Backstage emitted none of these numbers. Everything on that dashboard came from OpenTelemetry.

Let's build it again from zero, in twenty minutes, on this laptop.

Amsterdam, March 2026: 'is it up?'

Backstage CON NORTH AMERICA



The picture we drew last time: every plugin logs and counts its own way, nothing correlates

1. **instrumentation.js + --require** NodeSDK and auto-instrumentations, loaded before any other import
2. **Twelve built-in meters** catalog_entities_count, scaffolder.task.duration, backend_tasks.*
3. **A forced 500, a trace, an alert** 'the plugin returned 500, here is the span' in Grafana

The question we could not answer: is anyone using the portal, where do they get stuck, and which plugin burns the budget? That dashboard could tell you a request failed. It could not tell you whether the portal was worth running.

What Backstage emits natively: one slide

Backstage | CON NORTH AMERICA

```
# catalog
catalog_entities_count{kind}
catalog_registered_locations_count
catalog_relations_count
catalog.processed.entities.count{result}
catalog.processing.duration
catalog.processing.queue.delay
catalog.stitching.*
# scaffolder
scaffolder.task.count{template,result}
scaffolder.task.duration
scaffolder.step.count / .duration
# scheduler
backend_tasks.task.runs.count / .duration
```

Backstage 1.56; all via @opentelemetry/api; nothing starts the SDK for you.

Nothing for the things a product team asks about

- **HTTP per plugin** no request count, no latency, no error ratio
- **Search** no query count, no zero-result rate
- **TechDocs** no page views, no 404s
- **Permissions** allow / deny decisions are not metered
- **Users** no sign-ins, no active users, no sessions

The wiring itself: Mithun Banerjee's lightning talk at 5:15 today is the copy-paste version of instrumentation.js. I am skipping it. Today is about what to do with the data.

A portal is a product. Measure it like one.

Adoption

Who uses it, for what, how often. Active users, page views per plugin, templates run, hours saved.

Friction

Where people get stuck. Failed template runs, zero-result searches, links to entities that no longer exist.

Reliability per plugin

Which plugin burns the error budget. RED per plugin, slowest routes, the database behind each one.

Spotify measures time to the tenth merged PR and search success. **Insights** and Red Hat's **Adoption Insights** exist, but they are plugins with their own database.

Today: the same answers from **OpenTelemetry you already run**, with no new database and no product metric emitted by Backstage.

Four layers, each gives what the others cannot

Backstage | CON NORTH AMERICA

0 · Kernel

OBI, eBPF

Traces with Postgres per table, RED, Node event loop, GC and heap, network flows. Works on plugins you do not own.

```
one privileged container
zero code, no restart
```

1 · Runtime

Node SDK

In-process spans (express, pg), winston logs stamped with trace ids, Backstage's own twelve meters.

```
one env var
NODE_OPTIONS=--require ...
```

2 · Product

Auditor service

Who did what, and did it work: entity lookups by user, template runs by template, failures.

```
one config line
severityLogLevelMappings.low
: info
```

3 · User

AnalyticsApi

navigate, click, search with result count, create with time saved. The adoption numbers.

```
~90 lines, no SDK
one fetch every 2 s
```

Layer 0: OBI 1.0, zero code, no restart


```
obi:
  image: otel/ebpf-instrument:v1.0.0   # released 2026-10-30
  privileged: true
  pid: host
  environment:
    OTEL_EBPF_OPEN_PORT: "7007,5432"
    OTEL_EBPF_METRICS_FEATURES: application,application_runtime,network
    OTEL_EXPORTER_OTLP_ENDPOINT: http://otelcol:4318
  volumes: [/sys/kernel/tracing:/sys/kernel/tracing,
/sys/fs/cgroup:/sys/fs/cgroup]
```

What OBI logged when Backstage started:

```
instrumenting process cmd=/usr/local/bin/node pid=1395
    type=nodejs service=backstage
Script successfully injected  component=nodejs.Injector
# Backstage read: OTEL_SERVICE_NAME from its own env, nothing else
```

Injected through the inspector

SIGUSR1 opens the inspector, one Runtime.evaluate installs an AsyncLocalStorage bridge. Async-context propagation is marked Stable; event-loop, GC and heap metrics are experimental.

What you get for free

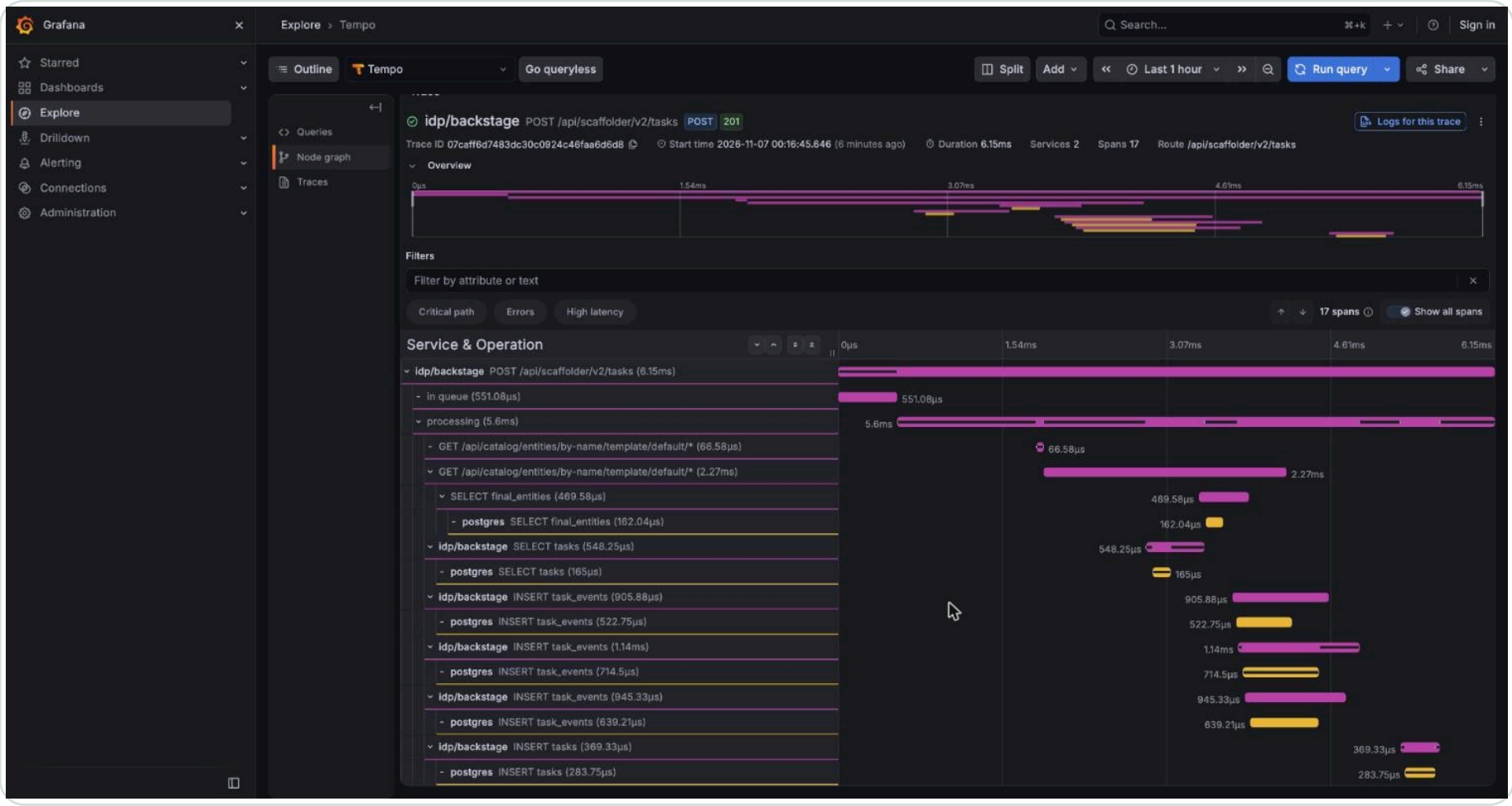
HTTP server and client spans (TLS included: static OpenSSL uprobes), Postgres statements per table, RED metrics with http.route, nodejs.eventloop.*, v8js.*, TCP flows.

What you do not get

Business attributes: template, entity, user. Logs as a signal. Header propagation on this linuxkit 7.0 kernel, and never into TLS egress.

One scaffolder POST, 17 spans, zero code

Backstage CON NORTH AMERICA



Tempo, layer 0 on the rig: the POST, the internal catalog call for the template, SELECT final_entities, INSERT tasks and task_events, each with the Postgres server span underneath

Layer 1: one environment variable


```
NODE_OPTIONS=--require @opentelemetry/auto-instrumentations-node/register
OTEL_EXPORTER_OTLP_ENDPOINT=http://otelcol:4318
OTEL_TRACES_EXPORTER=otlp
OTEL_METRICS_EXPORTER=otlp
OTEL_LOGS_EXPORTER=otlp
OTEL_METRIC_EXPORT_INTERVAL=10000
# runtime-node is OBI's job here; fs/dns/net are noise
OTEL_NODE_DISABLED_INSTRUMENTATIONS=runtime-node,fs,dns,net
```

Measured on the rig: container restart to ready in ~3 s; first meters in Prometheus within 10 s.

Gotcha. The winston instrumentation only ships log records when `@opentelemetry/winston-transport` is installed in the app. Without it you get trace ids in your log lines and nothing in Loki. Same story for the auditor's events.

Within ten seconds

1. **Backstage's own meters**
   scaffolder.task.count by template and result, catalog_entities_count by kind, scheduler runs

2. **Logs with trace ids**
   every winston line carries trace_id and span_id; audit events arrive as structured records

3. **In-process spans**
   express middleware, pg queries with the statement, undici calls, all nested under OBI's request

Product metrics are born in the Collector




```
processors:
  transform/plugin:
    trace_statements:
      - set(span.attributes["backstage.plugin"], span.attributes["url.path"])
          where IsMatch(span.attributes["url.path"], "^/api/[A-Za-z0-9_-]+")
      - replace_pattern(span.attributes["backstage.plugin"], "^/api/([A-Za-z0-9_-]+).*$", "$$1")
connectors:
  span_metrics:
    dimensions:
      - name: http.request.method
      - name: http.response.status_code
      - name: backstage.plugin
      - name: http.route
```

One fact, one dimension

Every backend plugin lives under /api/<pluginId>/. One regex turns that into a label on every span and, through span_metrics, on every RED series.

Same metric, any layer

In layer 0 the spans come from OBI, in layer 1 from the SDK. The dashboard does not care which; the Collector filters the duplicate.

Audit events become counters

Backstage | CON NORTH AMERICA

```
# one winston record per audit event (Backstage >= 1.36)
{ "isAuditEvent": true,
  "plugin": "scaffolder", "eventId": "task",
  "status": "succeeded",        # initiated | succeeded | failed
  "actor": { "actorId": "user:default/guest", ... },
  "meta":  { "actionType": "create",
             "templateRef": "template:default/paved-road-service" } }
```

Turn them on first

Severity 'low' maps to debug by default and the info-level logger drops it. One line: backend.auditor.severityLogLevelMappings.low: info.

actorId is a person

user:default/guest is PII once it is a Prometheus label. Hash it in the Collector (one OTTL SHA256 line) or drop it before it leaves the host.

```
connectors:
  count:
    logs:
      backstage.audit.events:
        conditions: ['attributes["isAuditEvent"] == true']
        attributes:
          - key: plugin
          - key: eventId
          - key: status
          - key: backstage.actor      # from actor.actorId
          - key: backstage.template   # from meta.templateRef
```

What that buys you: template runs by template and status, **who** ran them, entity lookups by query type, and the ghost lookups: fetches of entities that no longer exist.

No code in Backstage knows these metrics exist.

The browser talks OTLP: 90 lines, no SDK


```
captureEvent(event: AnalyticsEvent) {
  const attrs = {
    'backstage.analytics.action': event.action,     // navigate | click | search | create
    'backstage.plugin': event.context.pluginId,
    'enduser.id': this.userRef,                      // from identityApi
    'session.id': this.sessionId,
  };
  if (event.value !== undefined) attrs['backstage.analytics.value'] = event.value;
  if (event.action === 'search') attrs['backstage.search.zero_results'] = String(event.value ===
0);
  this.queue.push(otlpLogRecord(event, attrs));  // flushed every 2 s to :4318/v1/logs
}
```

Registered with `AnalyticsImplementationBlueprint` from @backstage/plugin-app-react. No OTel SDK in the browser: the module writes OTLP/JSON by hand and POSTs it.

Requested in backstage#25045 in 2024, closed not-planned. It is sixty lines.

Collector side

otlp receiver http with cors.allowed_origins for the portal origin. count connector turns actions into backstage_analytics_events_total.

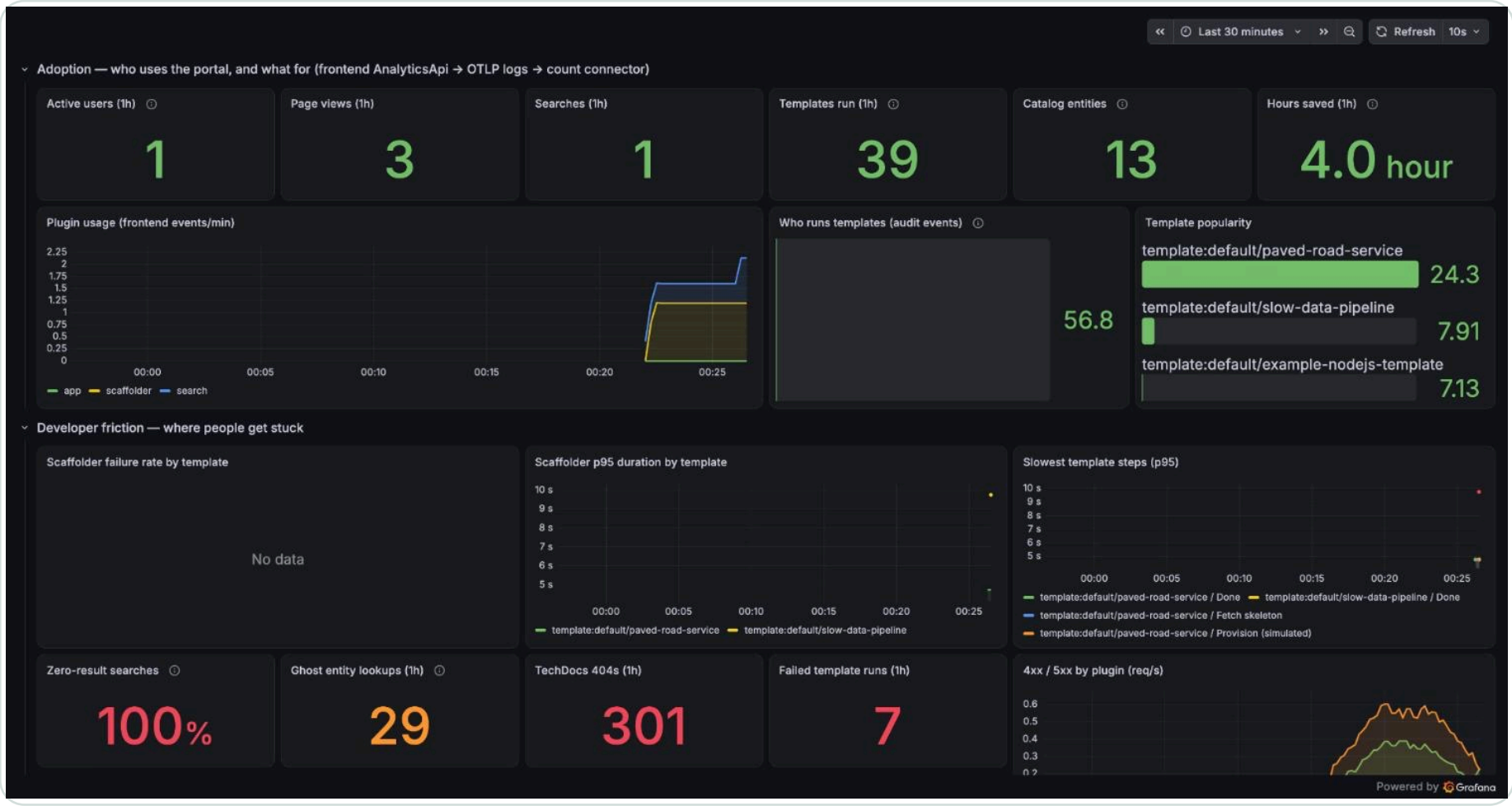
search carries the result count

value == 0 becomes backstage.search.zero_results = true. The zero-result rate is one PromQL division.

create carries time saved

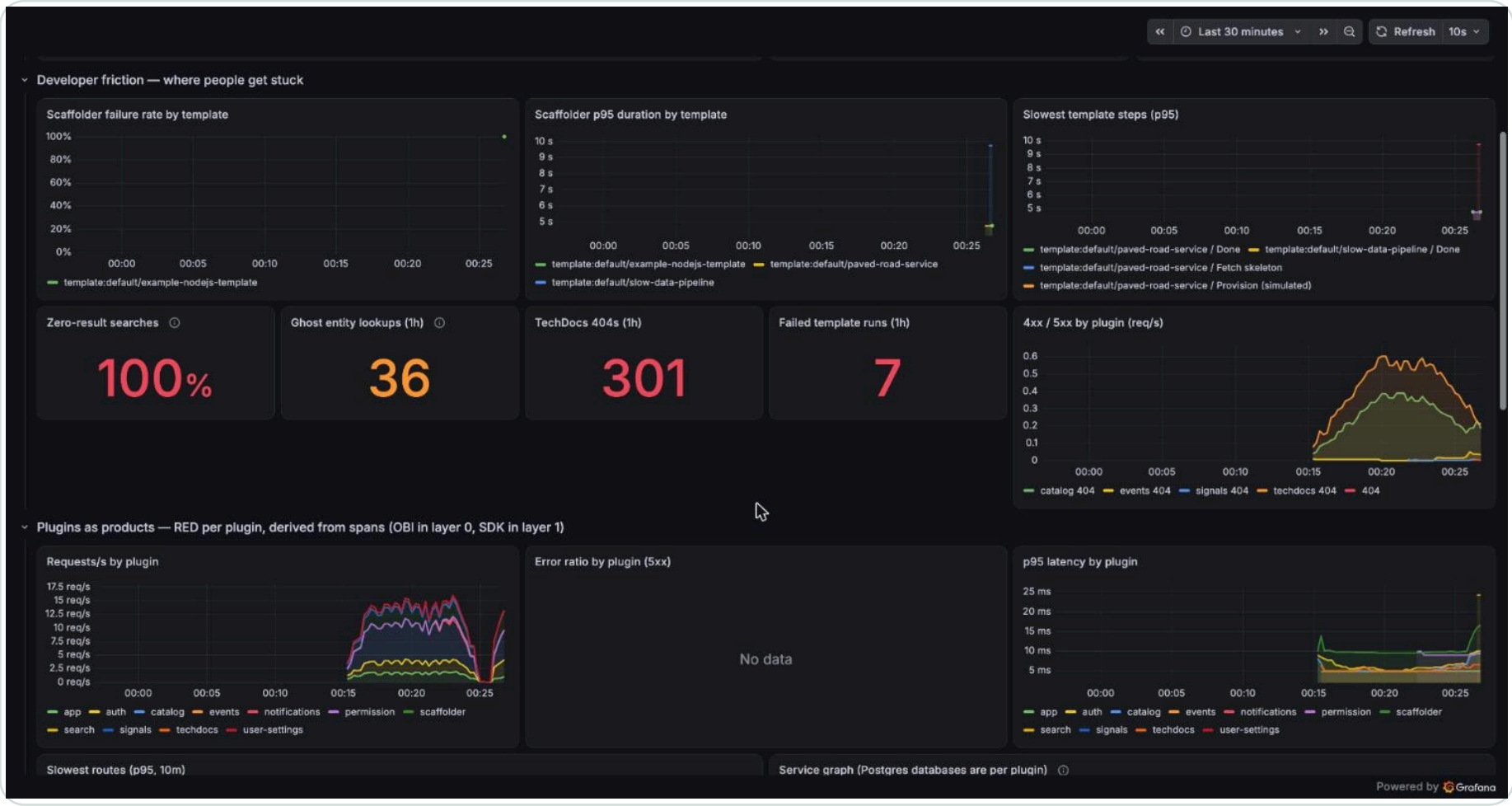
The scaffolder emits create with value = minutes from backstage.io/time-saved. Sum it in Loki: hours saved per template.

Adoption: who uses the portal, and for what



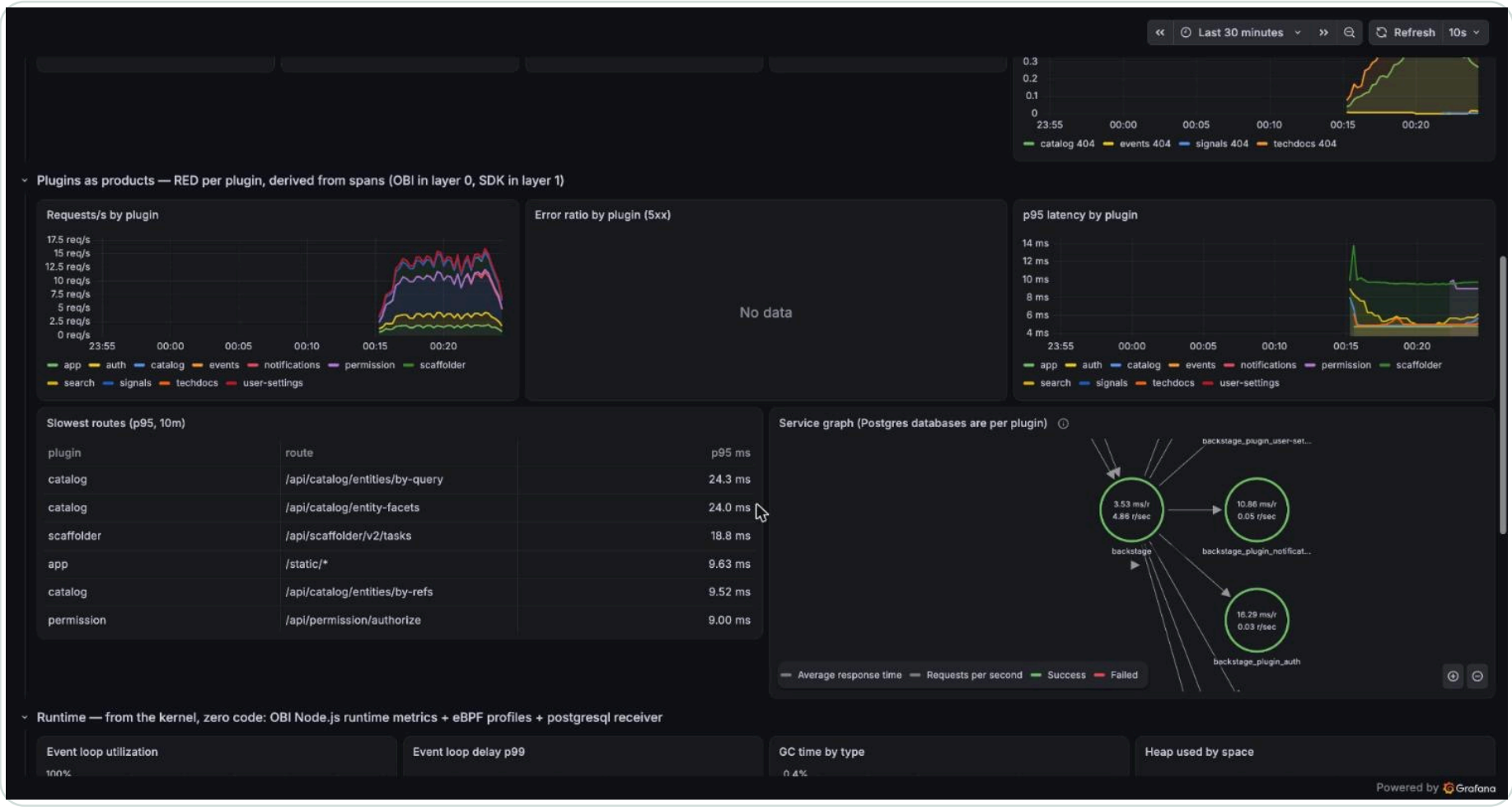
Dashboard row 1 on the rig: active users, page views, searches, templates run, hours saved, plugin usage, who runs templates (audit), template popularity

Friction: where people get stuck



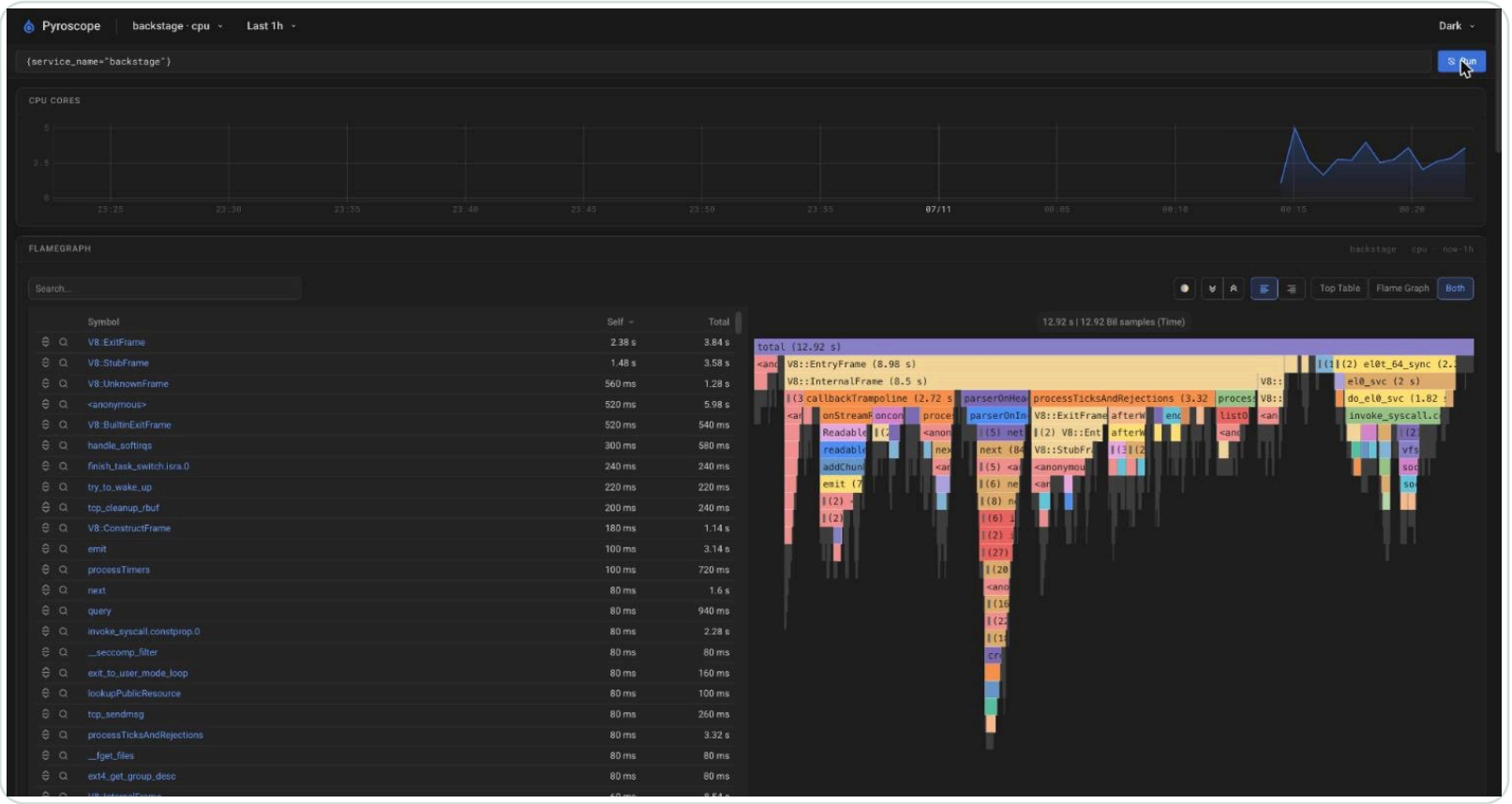
Dashboard row 2 on the rig: 100% failure on the stock GitHub template, p95 by template, slowest steps, zero-result searches, ghost lookups, TechDocs 404s

Plugins as products: RED per plugin



Dashboard row 3 on the rig: requests/s, error ratio and p95 per plugin, slowest routes, and the service graph with one Postgres database node per plugin

Fourth signal: a CPU profile, no agent



Pyroscope 2.4, OTel eBPF profiler distro 0.164: JS frames symbolized next to kernel frames

Profiles are OTel's fourth signal

Public alpha since March. The eBPF profiler runs as a Collector distro, exports OTLP profiles, Pyroscope ingests them natively.

V8 frames, not just native

Ignition, Sparkplug and TurboFan frames resolve to function names: net.Socket.write, ajv compileSchema, processTicksAndRejections.

Limit

Asynchronous stack traces are not built; you see where CPU goes, not which request asked for it.

Demo: four beats, one laptop

A — **Layer 0: the kernel** — zero code

make layer0 → OBI log 'Script successfully injected' → Tempo waterfall of a scaffolder POST → RED by plugin

B — **Layer 1: one env var** — ~10 s

make layer1 → ~3 s restart → scaffolder.task.count by template appears → a log line with trace_id

C — **Product metrics** — ~20 s

run a template in the UI → the create event lands with time saved → audit counters: who ran what

D — **Plugins as products** — dashboard

RED table per plugin → slowest routes → service graph with one database per plugin → flame graph

Trade-offs: what bit us on the rig

What	Why it bites	What we did
Two sources of the same span	OBI and the SDK both emit HTTP spans; RED doubles	pick an owner per layer; filter the other in the Collector (telemetry.distro.name)
Kernel caveats	linuxkit 7.0 disabled OBI header propagation; TLS egress never carries context	accept leaf spans for GitHub calls; mount /sys/kernel/tracing; read the OBI log
Audit events are debug by default	severity 'low' is dropped by the info-level logger	backend.auditor.severityLogLevelMappings.low: info
actorId is PII	user refs become Prometheus labels	hash with one OTTL SHA256 statement, or keep actors in Loki only
Looking costs money	~250 spans/s, ~150 log records/s on an 8 rps portal	put the Collector's own export rate on the dashboard; sample traces, never the counters

Starter SLOs for a developer portal

SLO	SLI source
Paved-road template success >= 97 %, p95 < 3 min	scaffolder.task.count/duration{template} (built-in)
Catalog read p95 < 500 ms, 5xx < 0.1 %	span_metrics per plugin
Search zero-result rate < 20 %	frontend search event value
Ghost-entity lookups trending to zero	auditor entity-fetch status=failed
TechDocs 404 rate < 2 %	span_metrics techdocs
Catalog freshness p95 catalog.processing.queue.delay < 5 min	built-in

Proposed, not published: no Backstage SLOs exist in the docs, Spotify's Insights or the CNCF talks we could find.

Take it home

Backstage | CON NORTH AMERICA

Everything you saw, on your laptop

```
git clone <repo>          # link below
make build && make up     # Backstage + OBI + Collector + Grafana
make load                 # synthetic developers, ~8 rps
make layer1               # SDK on; make layer0 = SDK off
```

- Collector config with the three derivations, the 36-panel dashboard, the AnalyticsApi module
- Three templates that tell the friction story: paved road, stock GitHub (fails), slow pipeline
- Pinned: Backstage 1.56.0 · OBI v1.0.0 · Collector contrib 0.164.0 · Pyroscope 2.4 · Grafana 13

repo QR

`github.com/<tbd>`



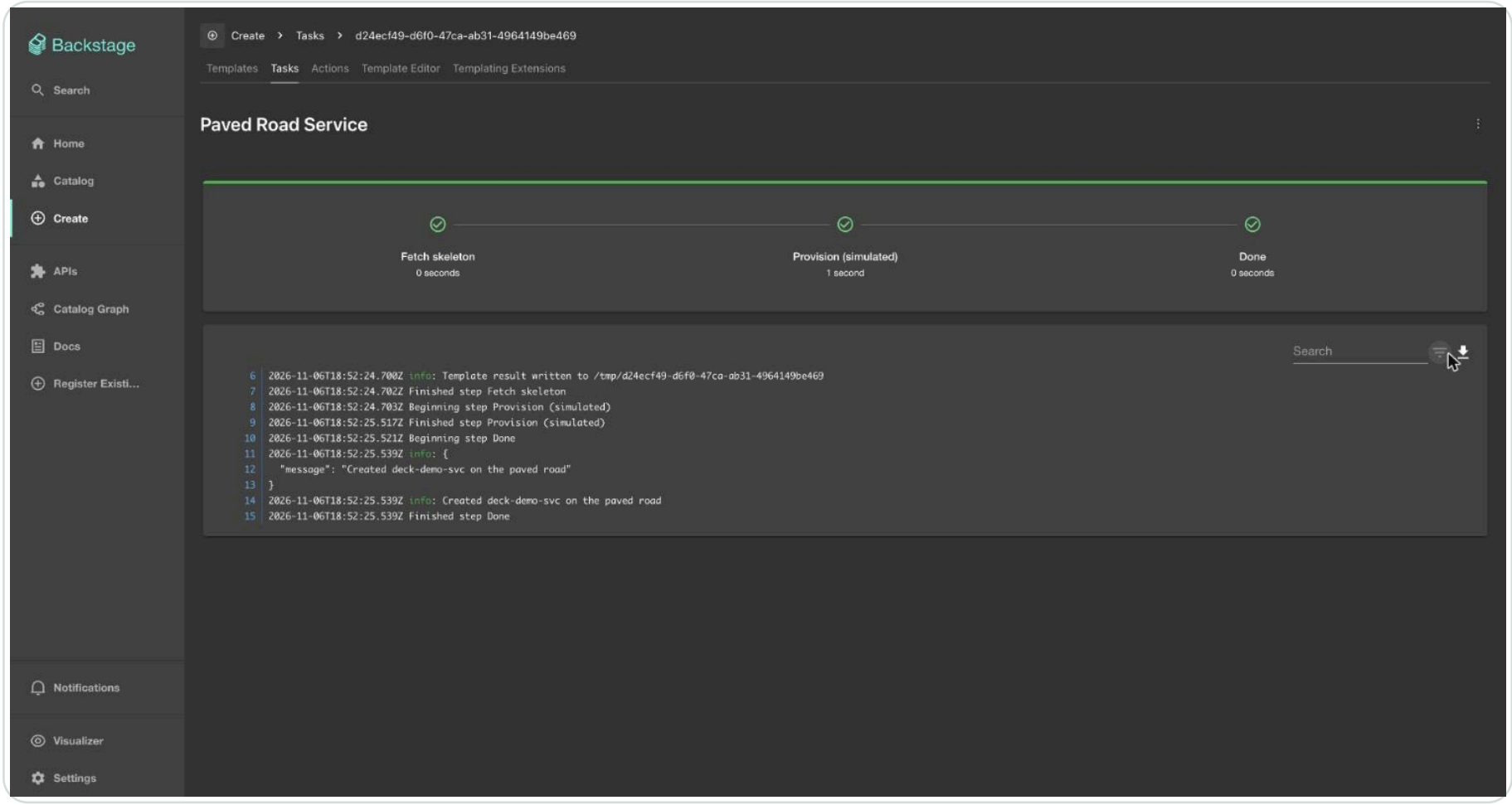
Backstage | CON NORTH AMERICA

Thank you · Questions?

Backup: the demo, frame by frame

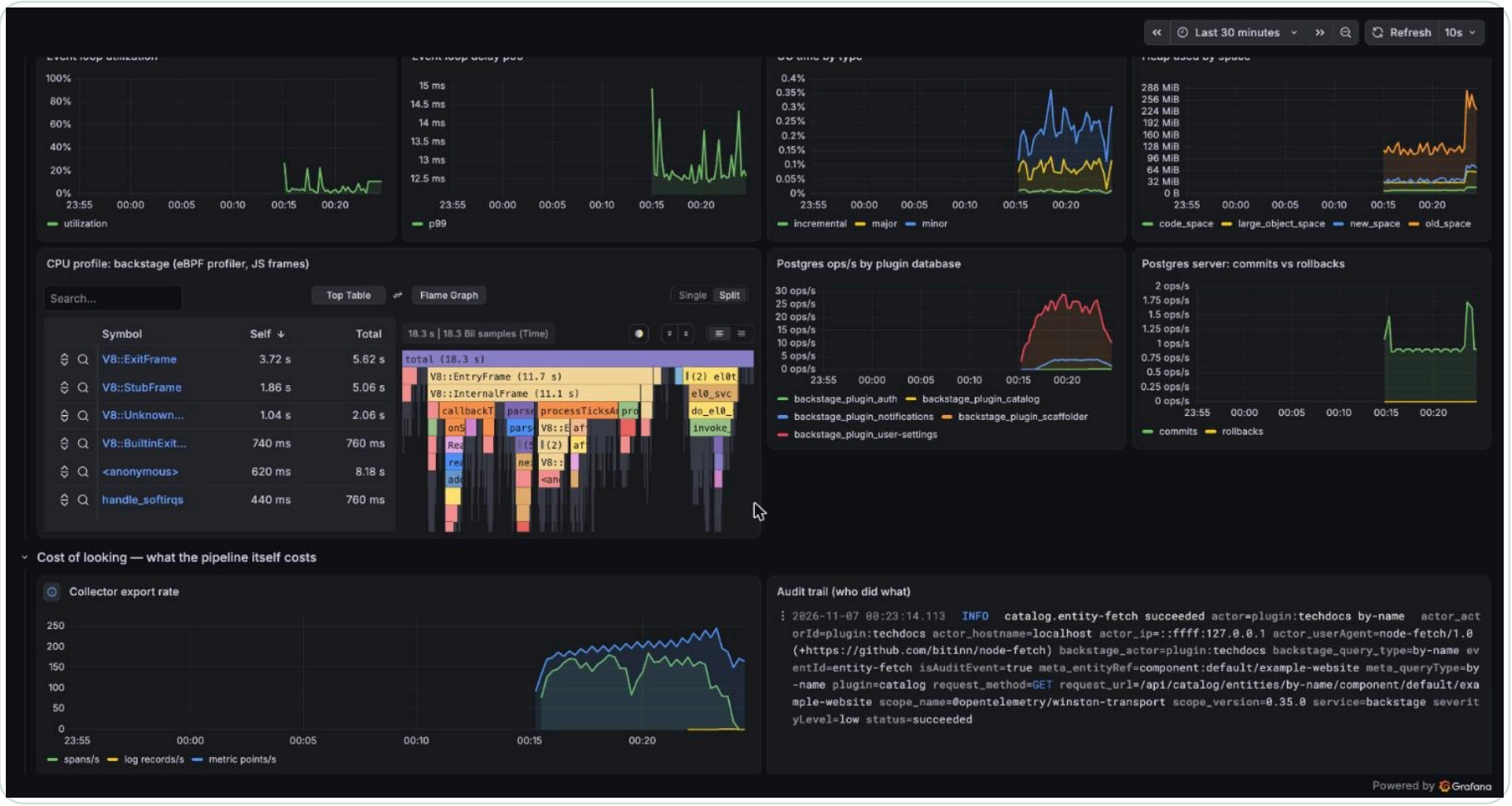
Captured on the rehearsal rig. Shown only if the live run misbehaves.

Beat C: a template run from the UI



Paved Road Service: fetch, provision (simulated), done. The frontend emits create with value 240 (PT4H) the moment the task is created.

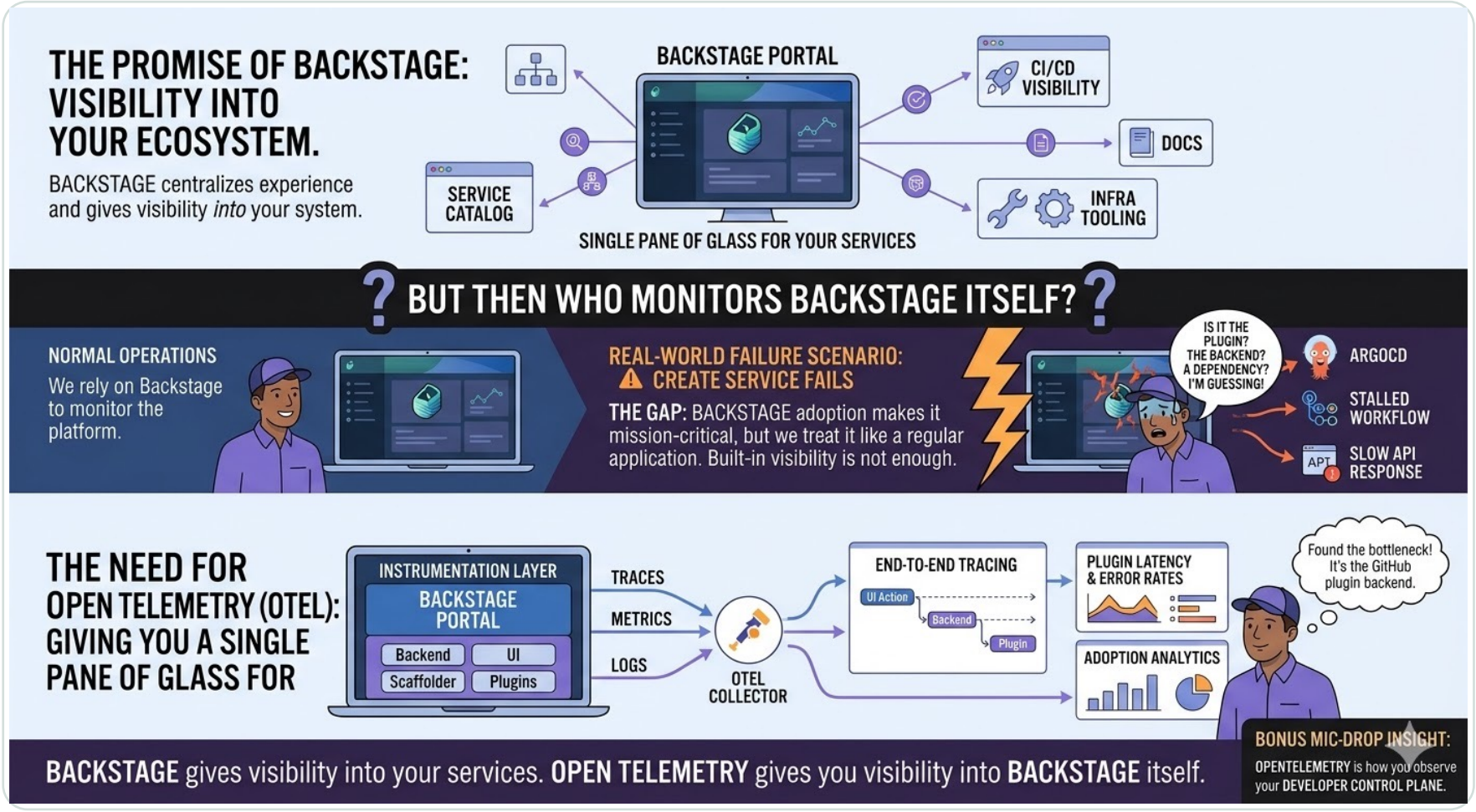
Beat D: the runtime row and the flame graph



Event loop utilization and p99 delay from OBI's injected agent, GC by type, heap by space, the flame graph, Postgres ops per plugin database, Collector export rate.

Amsterdam: who monitors the monitor?

Backstage CON NORTH AMERICA



The slide that opened the question last March.